

Chapter – 6

Dimension Reduction Method

Bal Krishna Nyaupane

Assistant Professor

Department of Electronics and Computer Engineering

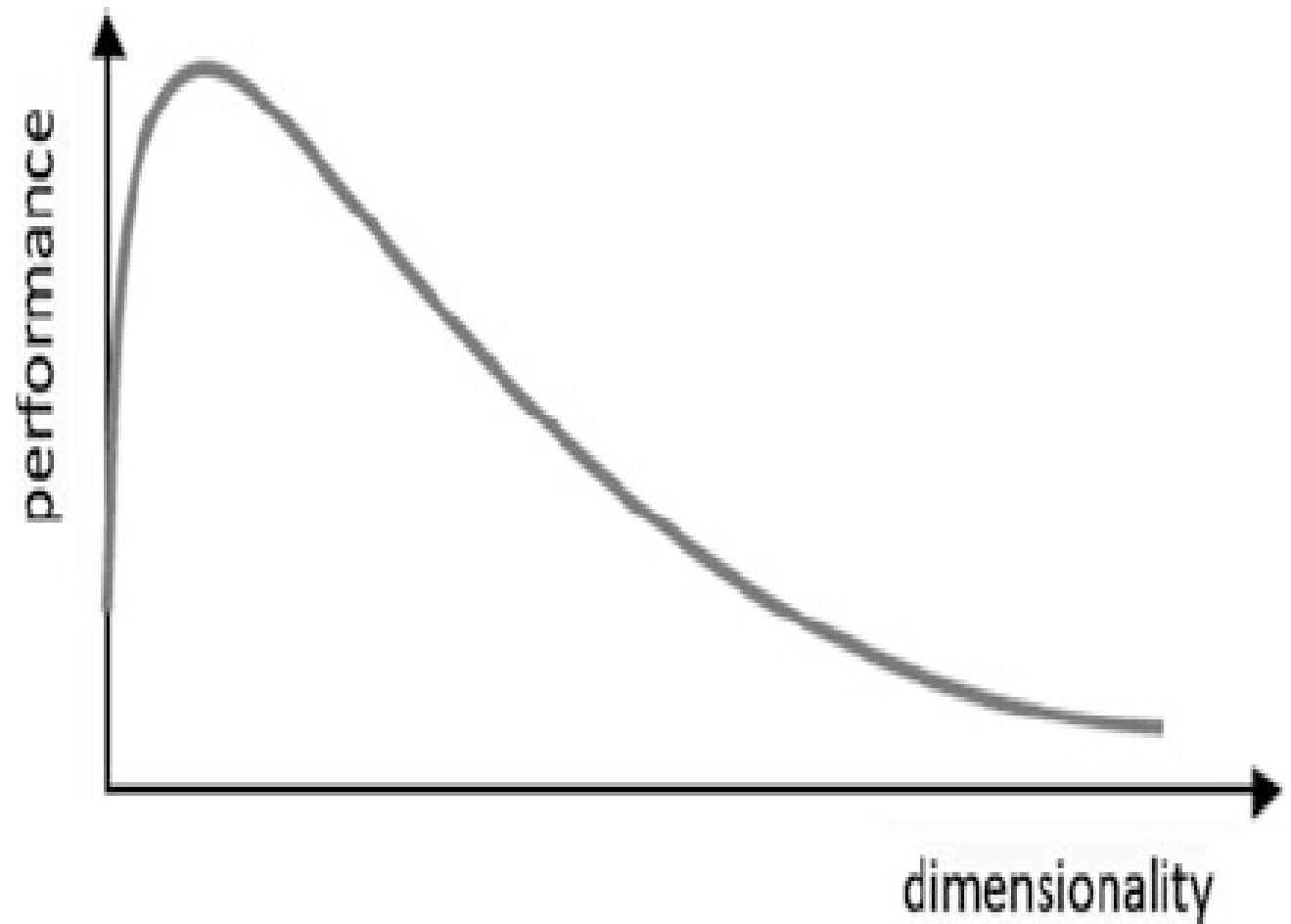
Paschimanchal Campus, IOE

bkn@wrc.edu.np

Introduction

- In most classification applications, the *complexity depends on the number of features*, d (which result in a d -dimensional feature space) and the *number of (training) data samples*, N .
- As the dimensionality (number of features) increases, increasing amounts of training data are required. This is called the **curse of dimensionality**.
- In order *to reduce the computational complexity*, the number of *features should be reduced to a sufficient minimum*.
- When data can be explained in terms of fewer features, the data can be analyzed visually more easily and we get a better idea about the underlying process.

Fig. 7.1 The performance of a classifier in terms of the dimensionality of the feature space



Introduction

- There are *two issues* that we must be careful about:
 - How is the classification *accuracy affected by the dimensionality* (relative to the amount of training data)?
 - How is the *complexity of the classifier* affected by the dimensionality?
- For a finite sample size, N , increasing the number of features will initially improve the performance of a classifier, but *after a critical value, further increase in the number of features (d) will reduce the performance* resulting in overfitting the data. *This is known as the peaking phenomenon.*

Problems of Dimensionality

- There are two main methods for reducing dimensionality
 - *Feature selection and*
 - *Feature extraction.*
- *In feature selection*, we want to select the k features (out of d) that provide the most information, discarding the other $(d-k)$ features.
- *In feature extraction*, we find a new set of k ($< d$) features which are combinations of the original d features. These methods may be supervised or unsupervised.
- The *most widely used feature extraction* methods are *Principal Components Analysis (PCA)* and *Linear Discriminant Analysis (LDA)*.

Principal Components Analysis (PCA)

- Principal component analysis (PCA) *is a standard tool in modern data analysis* - in diverse fields from neuroscience to computer graphics.
- Principal component analysis (PCA) is a statistical procedure that *uses an orthogonal transformation* to convert a set of observations of possibly correlated variables into a set of values of linearly uncorrelated variables *called principal components*.
- The number of principal components is *less than or equal to the number of original variables*.
- PCA allows us *to compute a linear transformation* that maps data from a *high dimensional space to a lower dimensional* sub-space.
- The goal of PCA is to *reduce the dimensionality* of the data *while retaining as much as possible* of the variation present in the original dataset.

Principal Components Analysis (PCA)

- It reduces the dimensions of a N-dimensional dataset by projecting it onto a (k)-dimensional subspace (where $k < N$) in order to increase the computational efficiency while retaining most of the information.

$$x = \begin{bmatrix} a_1 \\ a_2 \\ \dots \\ a_N \end{bmatrix} \longrightarrow \text{reduce dimensionality} \longrightarrow y = \begin{bmatrix} b_1 \\ b_2 \\ \dots \\ b_K \end{bmatrix} \quad (K \ll N)$$

Principal Components Analysis (PCA)

- Each principal component (PC1, PC2, . . .) is a linear combination of the original variables, and there are as many principal components as original variables.
- This transformation is defined in such a way that the ***first principal component has the largest possible variance*** and each succeeding component in turn has the next highest possible variance.
- Usually, the ***variance of the original data can be explained by the first few principal components*** and the ***rest can be ignored***.
- PCA is based on the sample covariance which ***characterizes the scatter of the entire data set, irrespective of class-membership***.
- The projection axes chosen by PCA ***might not provide good discrimination power***.

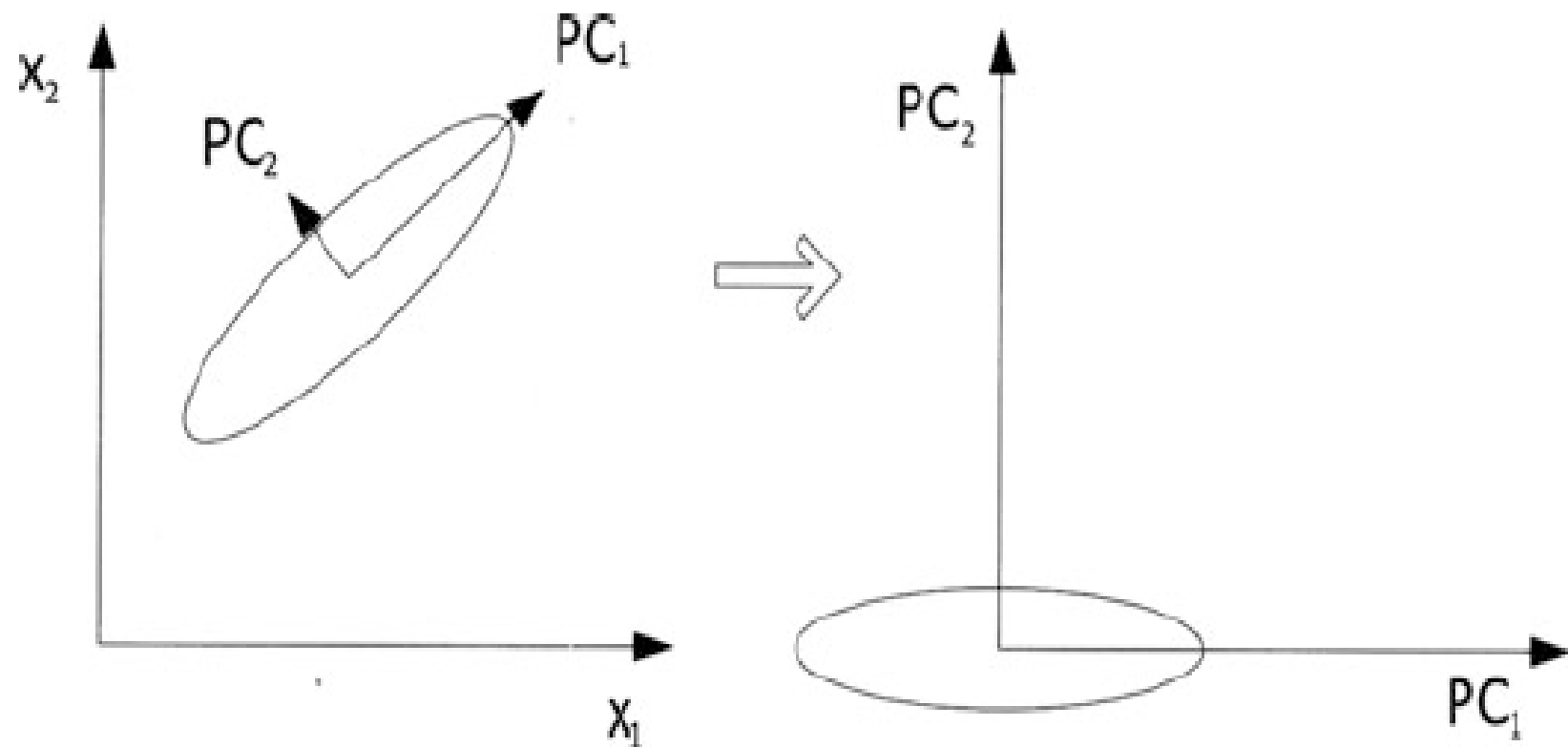
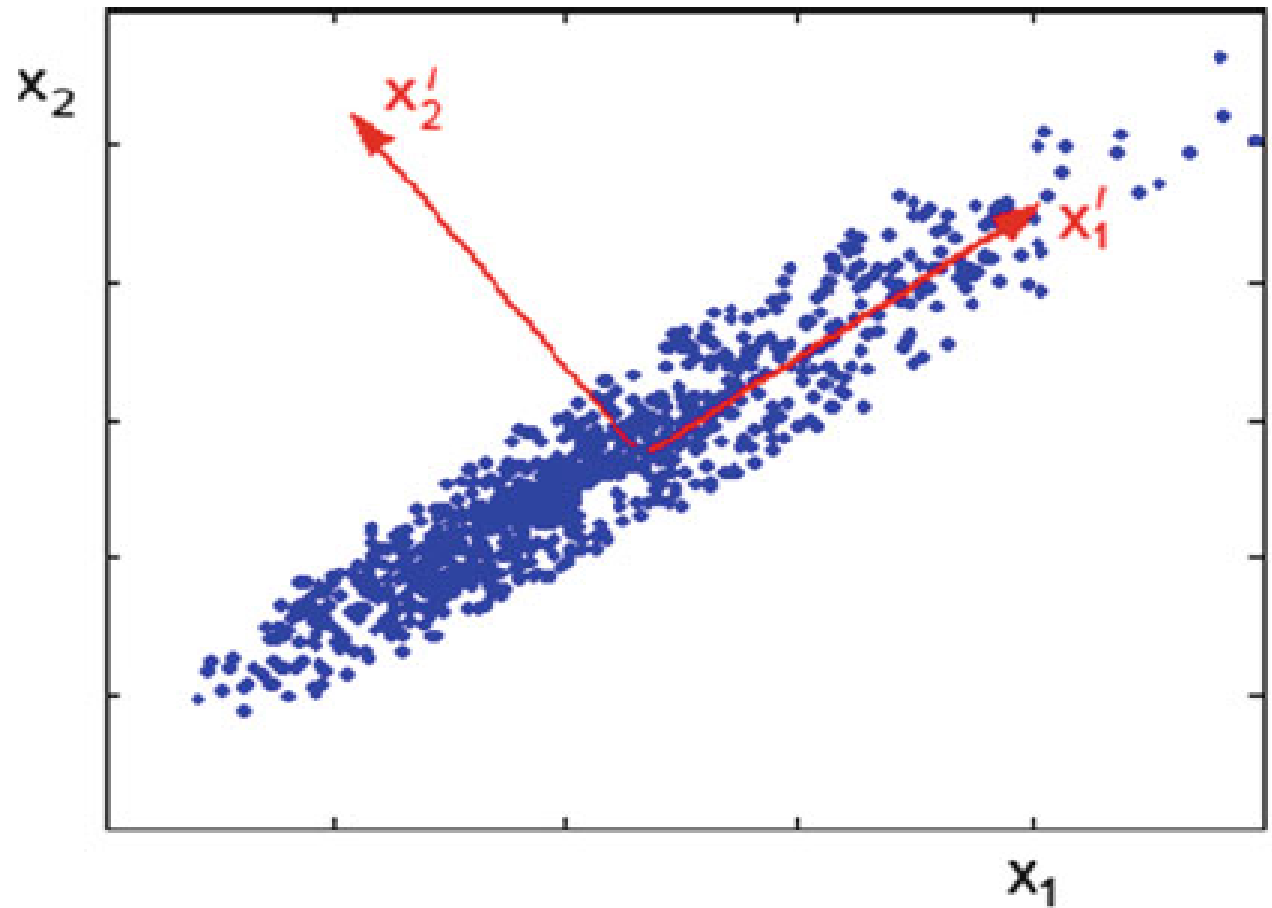
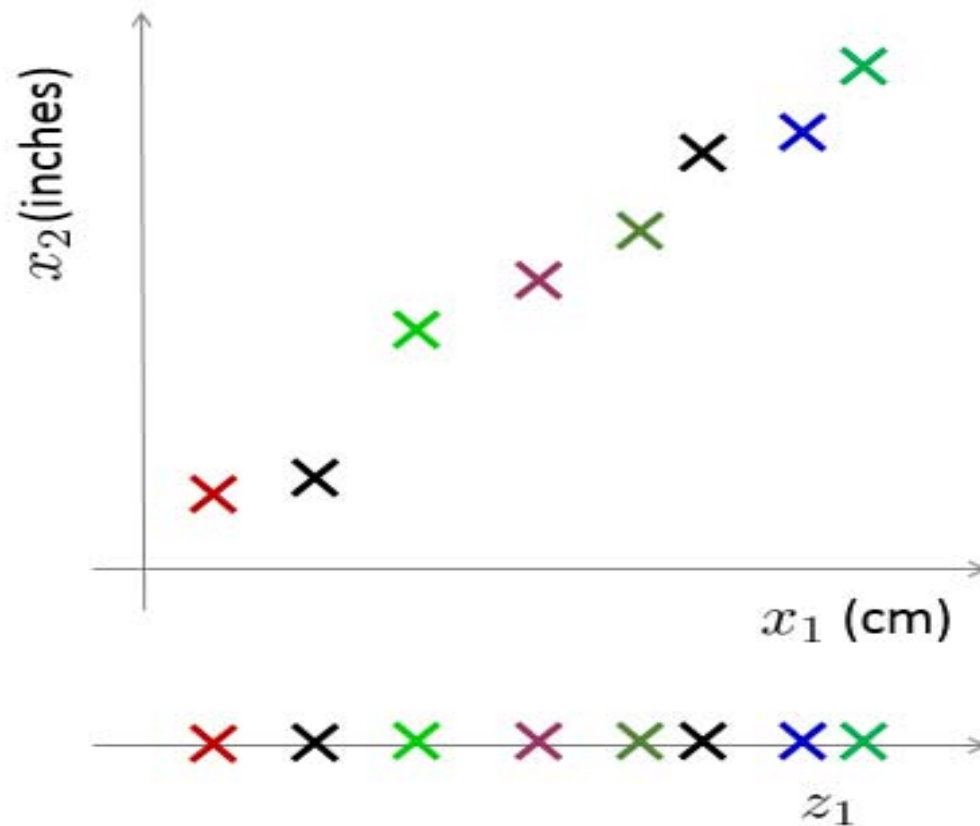


Fig. 7.4 Principal components analysis centers the data and then rotates the axes to line up with the directions of highest variance. If the variance on PCA2 is small, it can be ignored and the dimensionality reduces from two to one

Fig. 7.5 PCA in 2D: rotation of axes to line up with direction of largest variance (x'_1).



Most of the variance occurs in the x'_1 direction, and x'_2 can be ignored.



Reduce data from
2D to 1D

$$\begin{array}{rcl}
 x^{(1)} & \rightarrow & z^{(1)} \\
 x^{(2)} & \rightarrow & z^{(2)} \\
 & \vdots & \\
 x^{(m)} & \rightarrow & z^{(m)}
 \end{array}$$

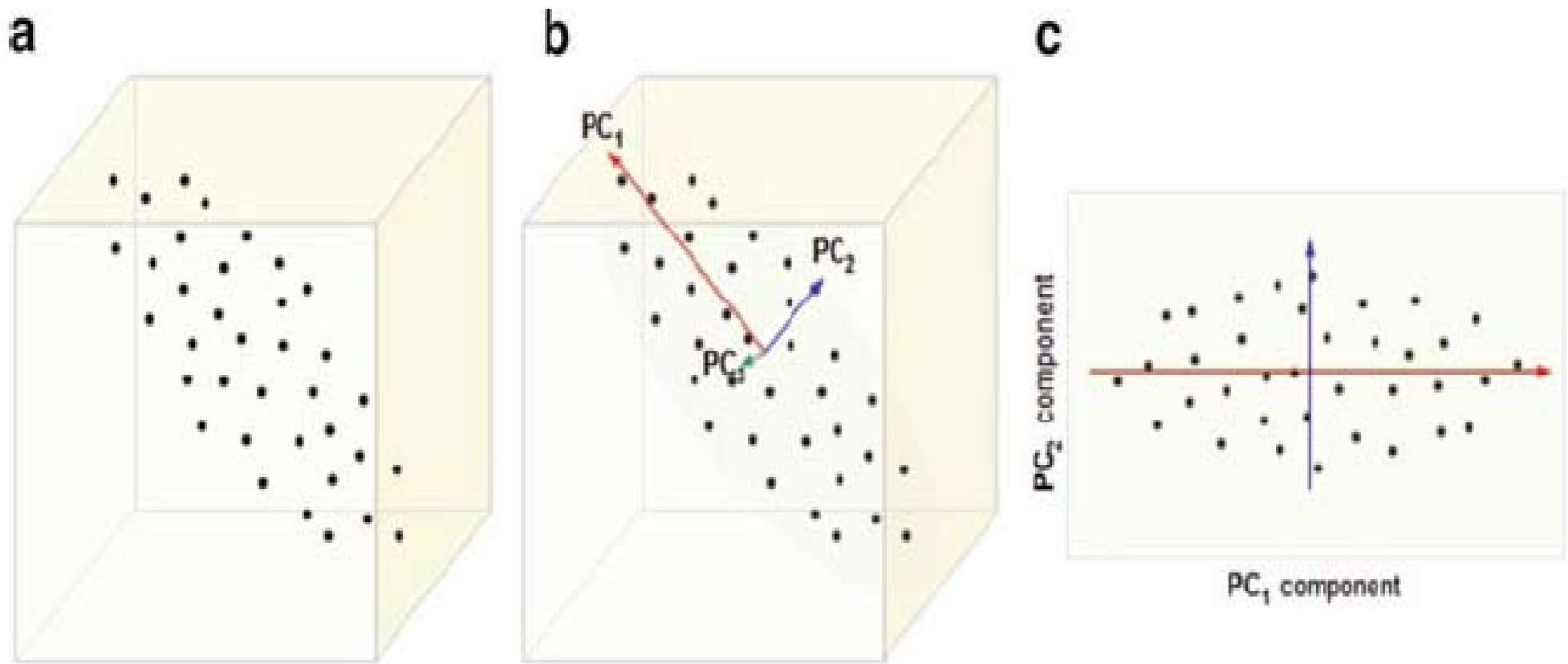


Fig. 7.7 (a) A dataset in 3D, (b) the principal component directions, (c) the two largest principal components

Principal Components Analysis (PCA)

- PCA can *reveal the internal structure of the data* in a way that best explains its variance, but it treats the total data and does not take into account class labels (i.e., it is unsupervised).
- There is *no guarantee that* the directions of *maximum variance will make good features for discrimination*.
- It is important to realize that principal components analysis *only helps if the original variables are correlated*.
- If they are *highly correlated*, there will be a *small number of eigenvectors with large eigenvalues*; a large reduction in dimensionality can be obtained by keeping only the k largest principal components.

PCA

- n is the Number of *features vectors*

$$X_1, X_2, \dots, X_n$$

- The *mean of these feature vectors* is given by $m = \frac{1}{n} \sum_{k=1}^n X_k$
- A *line passing through the mean of sample* given feature vectors *in the direction of e* , where e is an *unit vector passing through the mean m* , is given by

$$X = m + ae$$

- If $a=0$, $X=m$.
- For *different values of a* , moving from m in the direction of e .

$$X_k \approx m + ake$$



PCA

- The error function or the criteria function is defined by

$$J(a_1, a_2, \dots, a_n, e) = \sum_{k=1}^n \| (m + a_k e) - x_k \|^2$$

- This is the squared error within summation.

$$= \sum_{k=1}^n \| a_k e - (x_k - m) \|^2$$

$$= \sum_{k=1}^n a_k^2 \| e \|^2 - 2 \sum_{k=1}^n a_k e^t (x_k - m) + \sum_{k=1}^n \| x_k - m \|^2$$

- Here $\| e \|^2 = 1 = \| e^t e \|$. It is unit vector in the direction of the line.

PCA

- *To minimize the sum of squared error* by varying the values *of a*, *differentiate the error function* with respect *to a*, and *equate that to zero*.
- Take *gradient of J with respect to a* and equate that to 0.

$$\frac{\partial J}{\partial a} = 2\sum a_k - 2\sum e^t(X_k - m) = 0$$
$$a_k = e^t(X_k - m)$$

- This simply says that a_k *is orthogonal projection of* x_k *on to a line, the direction of e passing through the mean m.*
- So, simply what we get is the *sets of points*, this a_k represents these *projected point on to this line.*

PCA

- Now, need to *find the direction of the line*, that is e . it is determined that to find out a , it *should be orthogonal projection*.
- So, *write J* , the criteria function *as a function of e* . This *e is not error*. This is the *unit vector in the direction of the line*.

$$J(e) = \sum_{k=1}^n a_k^2 - 2 \sum_{k=1}^n a_k e^t (X_k - m) + \sum_{k=1}^n \|X_k - m\|^2$$

- we know $a_k = e^t (X_k - m)$

$$J(e) = - \sum_{k=1}^n a_k^2 + \sum_{k=1}^n \|X_k - m\|^2$$

$$= - \sum_{k=1}^n (e^t (X_k - m))^2 + \sum_{k=1}^n \|X_k - m\|^2$$

$$= - \sum [e^t (X_k - m) (X_k - m)^t e] + \sum \|X_k - m\|^2$$

PCA

- We know that if we *multiply co-variance matrix* by the number of *samples n* , we get the scatter matrix. *Scatter Matrix is*

$$S = \Sigma_{Scatter} = \sum [(X_k - m)(X_k - m)^t]$$

- The *co-variance tells about shape as well as size*, that means to *what extent that data is distributed* and whether the *data distribution is oriented in some particular directions*.
- *Scatter matrix gives the same information*, but with a scale factor of n .

PCA

- Then, $J(e) = -e^t S e + \sum \|x_k - m\|^2$
- To minimize criteria function $J(e)$ by varying e ,
 - Need to maximize the term: $-e^t S e$
 - $\sum \|x_k - m\|^2$, This term is independent of e .
- We know that $\|e\|^2 = e^t e = 1$
- Use of Lagrangian,

$$u = e^t S e - \lambda (e^t e - 1)$$

$$\frac{\partial u}{\partial e} = 2S e - 2\lambda e = 0$$

 $Se = \lambda e$ This is an Eigen value expression.

e = an eigenvector of S and λ = eigenvalues of S .

PCA

- $\sum \|X_k - m\|^2$, This is Constant term.
- *To maximie the term $-e^t S e$, we have to take the maximum value of λ .*
- So, we *have to consider the Eigen vector corresponding to the maximum Eigen value*.
- Our ultimate aim is to get the value of a_k , that is what represents the point. So, that value of a_k is called the principal component.
- People also *called it K-L transformation*.
- If we *take only one Eigen vector*, corresponding to the largest Eigen value, then the corresponding principal component is a scalar one and *effectively d dimensional feature vector converting to one dimensional feature vector*.

Steps of PCA

- **Step 1: Get some data**
- **Step 2: Subtract the mean**
 - For PCA to work properly, you have to subtract the mean from each of the data dimensions. The mean subtracted is the average across each dimension. All the ***X values have $\bar{X}$ subtracted*** and ***Y values have $\bar{Y}$ subtracted*** from them. ***This produces a data set whose mean is zero.***
- **Step 3: Calculate the covariance matrix**
- **Step 4: Calculate the eigenvectors and eigenvalues of the covariance matrix**

Steps of PCA

- Step 5: Reduce dimensionality and form feature vector the eigenvector with the highest eigenvalue is the principle component of the data set.

- Step 6: Deriving the new data set

$$\text{FinalData} = \text{RowFeatureVector} \times \text{RowDataAdjust}$$

- **RowFeatureVector** is the matrix with the eigenvectors in the columns *transposed* so that the eigenvectors **are now in the rows**, with the most significant eigenvector at the top.
- **RowDataAdjust** is the mean-adjusted data *transposed*, i.e. the **data items are in each column**, with each row holding a separate dimension.
- **FinalData** is the final data set, with data items in columns, and dimensions along rows.

Steps of PCA

■ Reconstruction of original Data

$$\text{RowDataAdjust} = (\text{RowFeatureVector}^{-1} \times \text{FinalData})$$

- **RowFeatureVector**⁻¹ is the inverse of **RowFeatureVector**. The inverse of our feature vector is actually equal to the transpose of our feature vector. This is only true because the elements of the matrix are all the unit eigenvectors of our data set.

- The final expression is :

$$\text{RowDataAdjust} = (\text{RowFeatureVector}^T \times \text{FinalData}) + \text{OriginalMean}$$

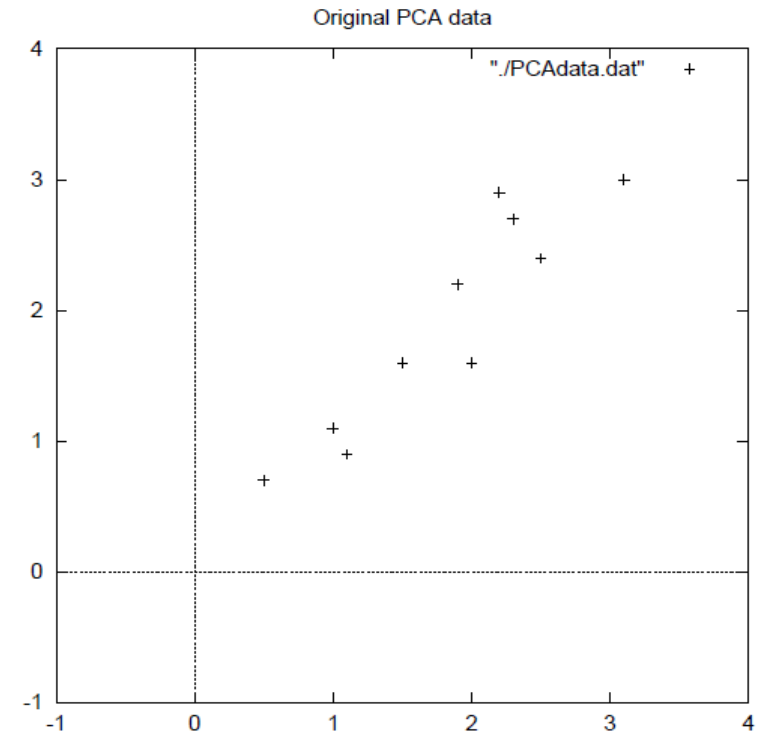
Steps of PCA: Example1

x	y
2.5	2.4
0.5	0.7
2.2	2.9
1.9	2.2
3.1	3.0
2.3	2.7
2	1.6
1	1.1
1.5	1.6
1.1	0.9

Data =

x	y
.69	.49
-1.31	-1.21
.39	.99
.09	.29
1.29	1.09
.49	.79
.19	-.31
-.81	-.81
-.31	-.31
-.71	-1.01

DataAdjust =



- PCA example data, original data on the left, data with the means subtracted on the right, and a plot of the data.

Steps of PCA: Example1

$$\text{Cov} = \begin{pmatrix} 0.6166 & 0.6154 \\ 0.6154 & 0.7154 \end{pmatrix}$$

$$\text{Eigenvalues} = \begin{pmatrix} 0.0491 \\ 1.2840 \end{pmatrix}$$

$$\text{Eigenvectors} = \begin{pmatrix} -0.7352 & -0.6779 \\ 0.6779 & -0.7352 \end{pmatrix}$$

- *It is important to notice that these eigenvectors are both unit eigenvectors.*

Steps of PCA: Example1

- *In fact, the* eigenvector with the *highest* eigenvalue is the *principle component of the data set*.
- In general, *once eigenvectors are found from the covariance matrix*, the *next step is to* order them by eigenvalue, *highest to lowest*.
- *Construct* the matrix with the eigenvectors *in the columns transposed* so that the eigenvectors *are now in the rows*, with the most significant eigenvector at the top.

$$\text{Eigenvectors} = \begin{pmatrix} -0.6779 & -0.7352 \\ -0.7352 & 0.6779 \end{pmatrix}^T$$

Steps of PCA: Example1

$$\text{FinalData} = \text{RowFeatureVector} \times \text{RowDataAdjust}$$

$$\text{FinalData} = \begin{pmatrix} -0.6779 & -0.7352 \\ -0.7352 & 0.6779 \end{pmatrix} * \begin{pmatrix} 0.69 & -1.31 & 0.39 \\ 0.49 & -1.21 & 0.99 \dots \end{pmatrix}$$

- The eigenvector with the highest eigenvalue gives the principle component of the data set

$$-0.6779 * 0.69 + -0.7352 * 0.49 = 0.82801$$

- The eigenvector with the *lowest eigenvalue gives the Lowest principle component* of the data set *that can be ignored.*

$$-0.7352 * 0.69 + 0.6779 * 0.49 = 0.17511$$

Steps of PCA: Example1

Transformed Data=

x	y
-.827970186	-.175115307
1.77758033	.142857227
-.992197494	.384374989
-.274210416	.130417207
-1.67580142	-.209498461
-.912949103	.175282444
.0991094375	-.349824698
1.14457216	.0464172582
.438046137	.0177646297
1.22382056	-.162675287

Steps of PCA: Example1

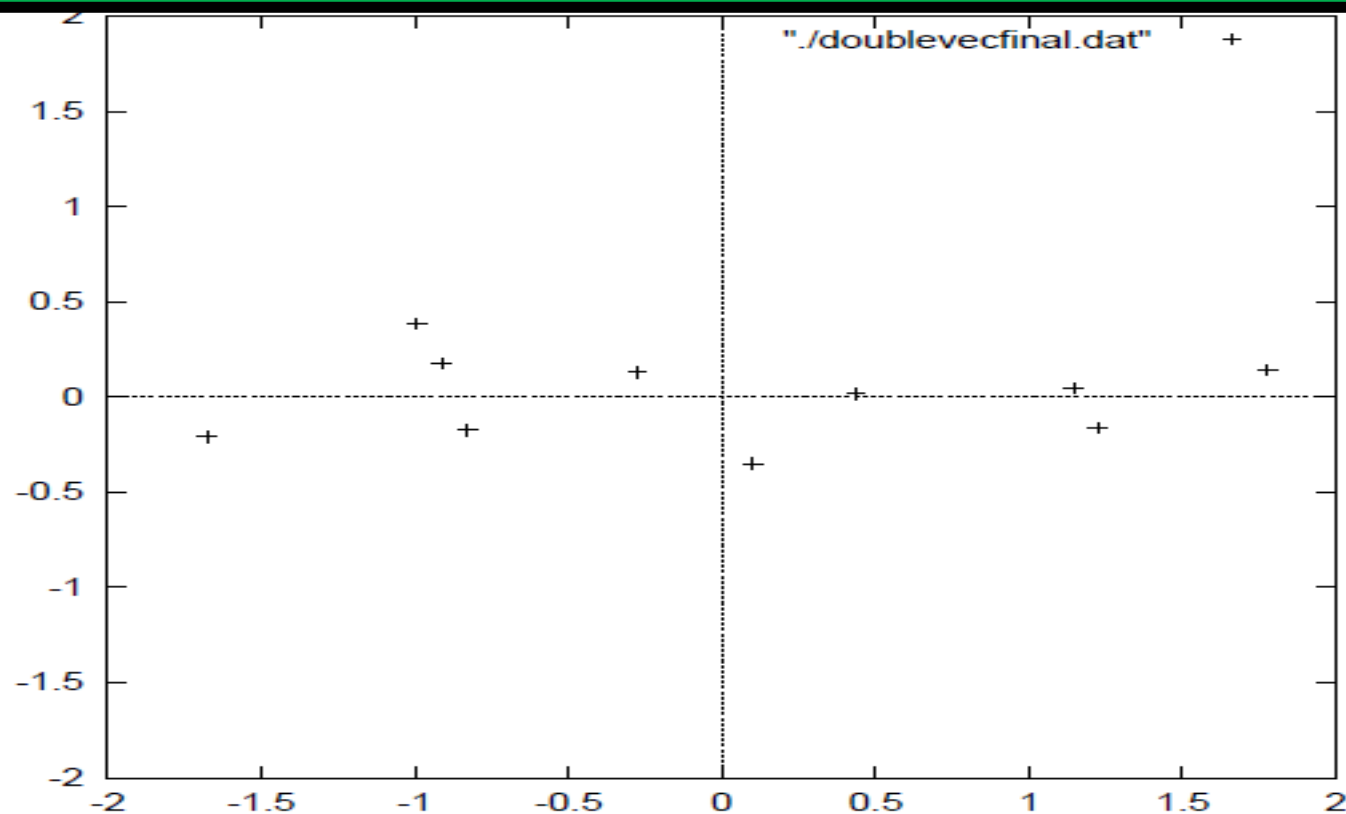


Figure 3.3: The table of data by applying the PCA analysis using both eigenvectors, and a plot of the new data points.

Steps of PCA: Example1

Transformed Data (Single eigenvector)

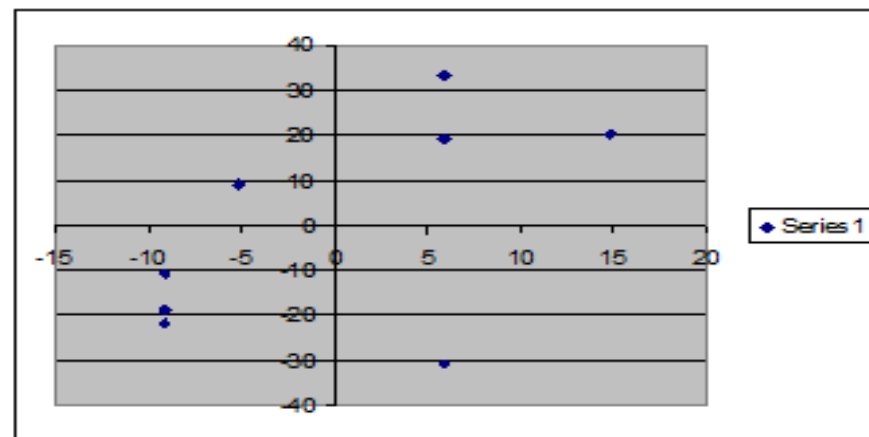
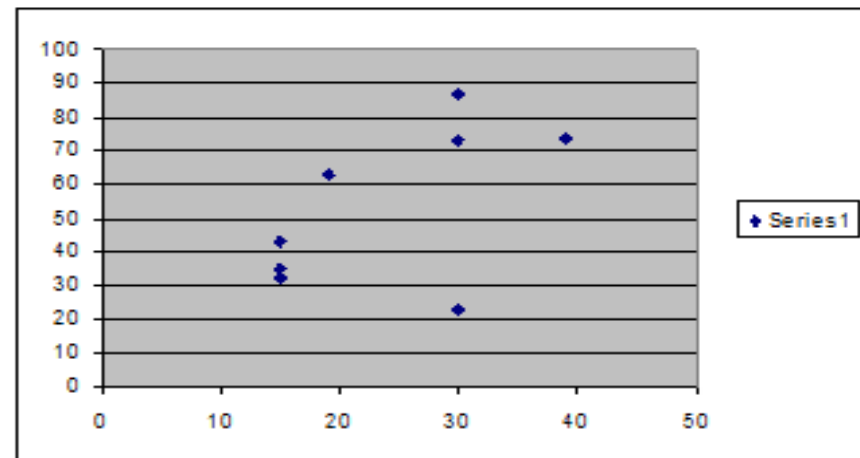
x
-.827970186
1.77758033
-.992197494
-.274210416
-1.67580142
-.912949103
.0991094375
1.14457216
.438046137
1.22382056

Figure 3.4: The data after transforming using only the most significant eigenvector

Steps of PCA: Example2

X1	X2	X1'	X2'
19	63	-5.1	9.25
39	74	14.9	20.25
30	87	5.9	33.25
30	23	5.9	-30.75
15	35	-9.1	-18.75
15	43	-9.1	-10.75
15	32	-9.1	-21.75
30	73	5.9	19.25

Mean1=24.1
Mean2=53.8



Steps of PCA: Example2

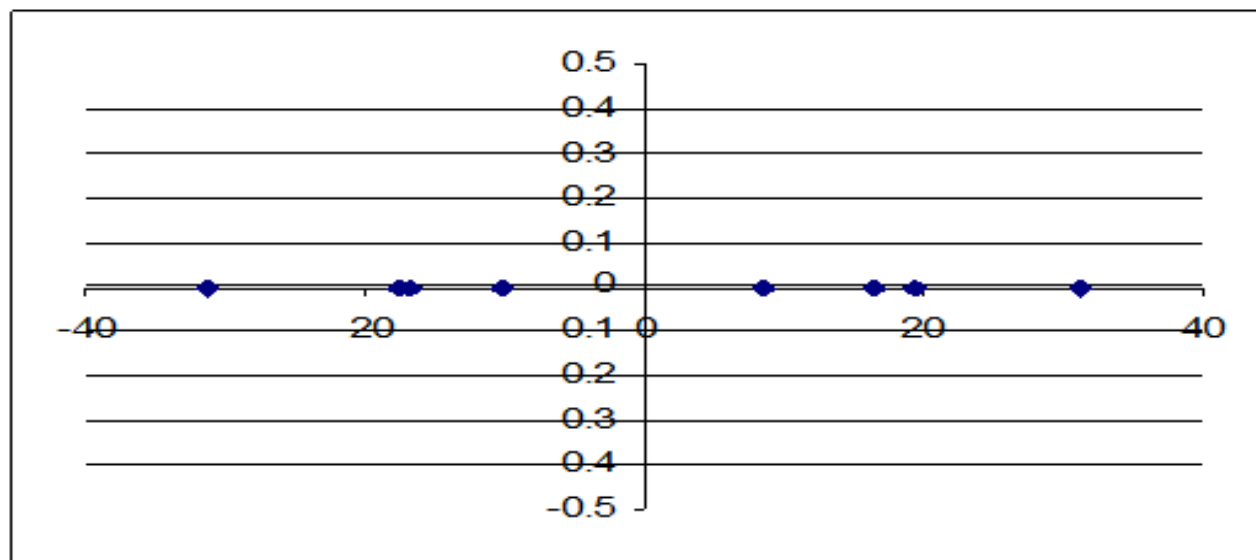
$$\text{Cov} = \begin{array}{|c|c|} \hline 75 & 106 \\ \hline 106 & 482 \\ \hline \end{array}$$

- We find out:
 - Eigenvalue and Eigenvectors:
 - $e1=(-0.98,-0.21), \lambda1=51.8$
 - $e2=(0.21,-0.98), \lambda2=560.2$
 - Thus the second eigenvector is more important!

Steps of PCA: Example2

- If we only keep one dimension: e2
 - We keep the dimension of e2=(0.21,-0.98)
 - We can obtain the final data as

$$y_i = (0.21 \quad -0.98) \begin{pmatrix} x_{i1} \\ x_{i2} \end{pmatrix} = 0.21 * x_{i1} - 0.98 * x_{i2}$$



yi	
	-10.14
	-16.72
	-31.35
	31.374
	16.484
	8.624
	19.404
16	-17.63

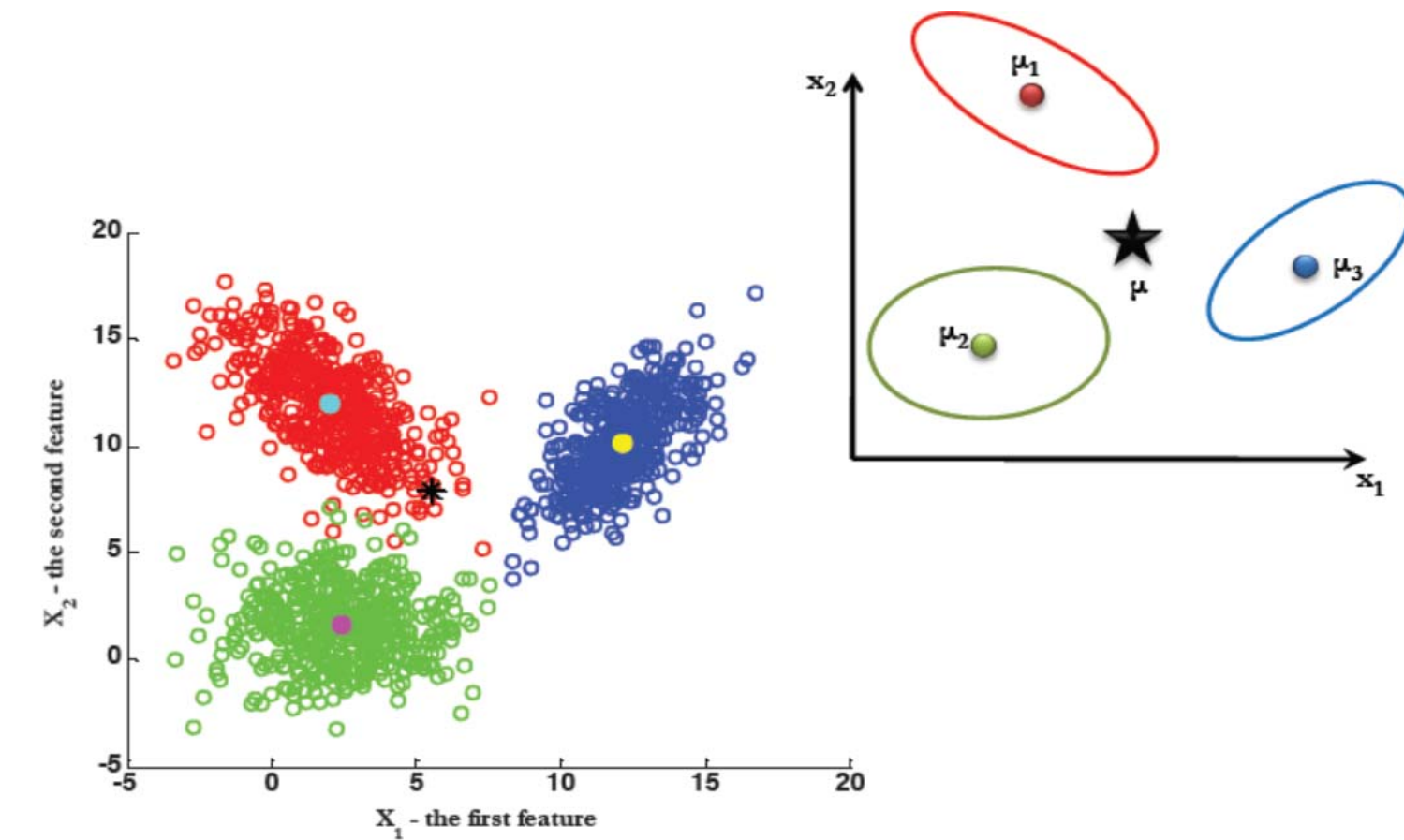
Linear Discriminant Analysis (LDA)

- Linear Discriminant Analysis (LDA) is used to solve dimensionality reduction for data with higher attributes
 - ***Pre-processing step*** for pattern-classification and machine learning applications.
 - Used ***for feature extraction***.
 - Linear transformation that ***maximize the separation between multiple classes***.
 - “***Supervised***” - Prediction agent

Linear Discriminant Analysis (LDA)

- What is the *goal of LDA*?
 - Perform dimensionality reduction while preserving as much of the class discriminatory information as possible.
 - Seeks *to find directions along* which the classes are *best separated*.
 - Takes into consideration the *scatter within-classes* but also the *scatter between-classes*.
 - LDA computes a transformation that *maximizes the between-class scatter* while *minimizing the within-class scatter*.
 - More *capable of distinguishing image variation* due to identity from variation due to other sources such as illumination and expression.

Linear Discriminant Analysis (LDA)



PCA Vs. LDA

- Both PCA and LDA are linear transformation techniques. However, PCA is an unsupervised while LDA is a supervised dimensionality reduction technique.
- PCA has no concern with the class labels. PCA tries to find the directions of the maximum variance in the dataset.
- In a large feature set, there are many features that are merely duplicate of the other features or have a high correlation with the other features. Such features are basically redundant and can be ignored.
- The role of PCA is to find such highly correlated or duplicate features and to come up with a new feature set where there is minimum correlation between the features or in other words feature set with maximum variance between the features.

PCA Vs. LDA

- Unlike PCA, LDA tries to reduce dimensions of the feature set while retaining the information that discriminates output classes.
- LDA tries to find a decision boundary around each cluster of a class. It then projects the data points to new dimensions in a way that the clusters are as separate from each other as possible and the individual elements within a cluster are as close to the centroid of the cluster as possible.
- The new dimensions are ranked on the basis of their ability to maximize the distance between the clusters and minimize the distance between the data points within a cluster and their centroids. These new dimensions form the linear discriminants of the feature set.

PCA Vs. LDA

■ **Case Study: PCA versus LDA**

- A. Martinez, A. Kak, "PCA versus LDA", *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 23, no. 2, pp. 228-233, 2001.

■ **Is LDA always better than PCA?**

- There has been a tendency in the computer vision community to prefer LDA over PCA.
- This is mainly because LDA deals directly with discrimination between classes while PCA does not pay attention to the underlying class structure.
- This paper shows that when the training set is small, PCA can outperform LDA.
- When the number of samples is large and representative for each class, LDA outperforms PCA.

Thank You

???

■ *References:*

- 1) R.O.Duda, P.E.Hart and D.G.Stork, Pattern Classification, John Wiley, 2001
1. S.Theodoridis and K.Koutroumbas, Pattern Recognition, Academic Press, 2009
2. C.M.Bishop, Pattern Recognition and Machine Learning, Springer, 2006
3. Kevin P. Murphy ,Machine Learning A Probabilistic Perspective